



Class and Object

1. Creating class
2. Creating object
3. Terminologies
4. Constructor
5. Destructor
6. Array of object

1. Creating class

- 1) Identify the real life entity to model
- 2) Do its abstraction of state as well as behavior
- 3) Map attributes of state using variables. Select most suitable data type and give proper names to variables
- 4) Map behavior by writing functions. Give appropriate name to the function. Decide necessary parameters and return type
- 5) Bind above variables (state) and functions (behavior) by writing them inside a class. Give appropriate name to the class that indicates the entity it is modeling on our program.
- 6) Decide what to hide from the code outside the class and make it private

```
class Person {  
    char name[64];  
    int age;  
public:  
    public char* getName() {  
        return name;  
    }  
    public void setName(char* str) {  
        strcpy(name,str);  
    }  
    public int getAge() {  
        return age;  
    }  
    public void setAge(int a) {  
        age=a;  
    }  
};
```

❑ Class v/s object

- Class is a template that models state and behavior of object. Object is a variable created from the class.
`int a; Person p1;`
What `a` is for built-in data type “`int`”, `p1` is for user-defined class “`Person`”
- First class is created. Then objects are created from class
- Object creation is called as object instantiation
- Resources are not allocated when class is created. They are allocated to objects that are created from class

❑ Creating object

- `Person p1;` // Object `p1` is created on stack
- `p1.getName();` // method `getName` invoked on object `p1`
- `Person* ptr= new Person();` // Object created on heap and pointed by pointer `ptr` on stack
- `ptr->getName();` // method `getName` is invoked using pointer `ptr`

- Entity
- Class
- Object
- Instance and instantiation
- Class members
 - Data members
 - Member functions
 - Accessors
 - Mutators
- Invoking object
- Parameters v/s arguments

- What is object initialization. Examples of initialization
- Constructor: A member function that initializes object
- Called by compiler, immediately after creation of object
- Rules:
 - Name same as class
 - No return type
- Types:
 - Default constructor
 - Parameter-less constructor
 - Parameterized constructor

- Constructor overloading
 - Need
 - Same name with different signature
 - Signature is number, type and sequence of parameters.
- Compiler selects definition based on best match of parameters with arguments

- What is object un-initialization. Examples of un-initialization
- Destructor: A member function that un-initializes object
- Called by compiler, just before removing the object from memory
- Rules:
 - Name same as class with ~
 - No return type
- Types:
 - Destructors can't be overloaded as they don't have parameters

- Syntax
Person p1[5]; // 5 objects of class Person created on stack
Person * ptr= new Person[5]; // 5 objects of class Person created on heap, pointed by ptr
- All members initialized by default / parameter-less constructor only.
- Hence, class must have default constructor or parameter-less constructor
- Elements accessed by index, starting from 0
p1[0].getName(); // Sending message to first element from array p1 on stack

ptr[0].getName(); // Sending message to first element from array p1 on heap